



[News](#)
[Features](#)
[Documentation](#)
[Application examples](#)
[FDTD tutorial](#)
[Download](#)
[Participate](#)

Convolutional perfectly matched layer boundary condition

Summary

We will setup cubic volume with Convolutional Perfectly Matched Layer (CPML) boundary condition and a point source.

What can be tested:

- compare performance of different CPML settings
- compare performance of Liao boundary condition (modify paramtere BOUNDARY_ALL to liao)
- try to add some material object to the computational volume, including boundaries

Setup all this with XSvit

Note that some of the screenshots might be from an older version of XSvit.

Start the XSvit application and setup the computational volume in this way:

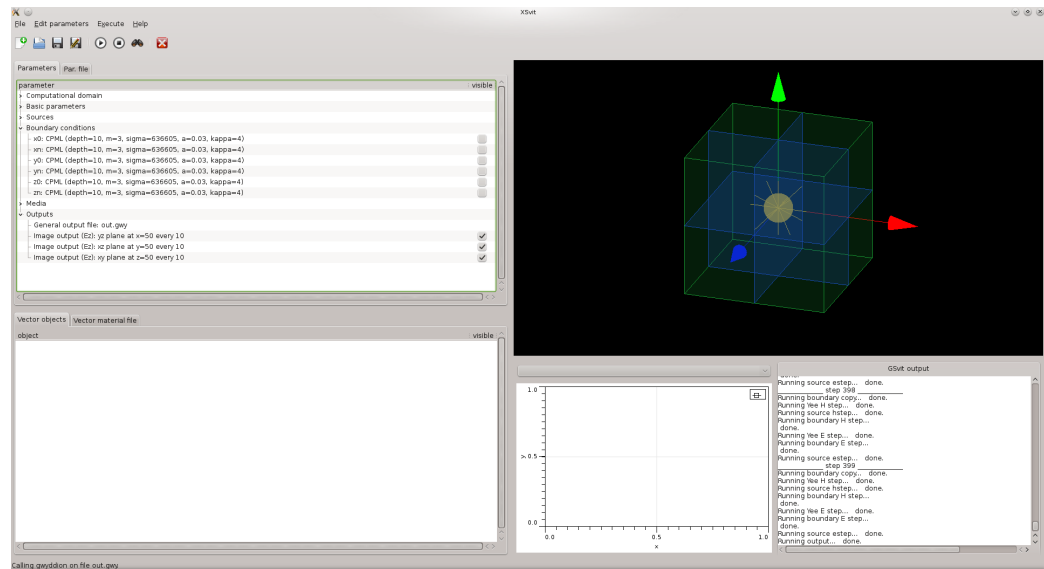
- Computational volume size 100x100x100 voxels, discretisation 1e-8, 1e-8, 1e-8.
- 400 computation steps
- Point source with orientation angles (theta and phi) set to zero, sine wave with amplitude 1V, wavelength 200 nm.
- Ez image outputs in every normal direction, at center of the computational volume.

We refer to [basic application use example](#) for details how to adjust all these settings

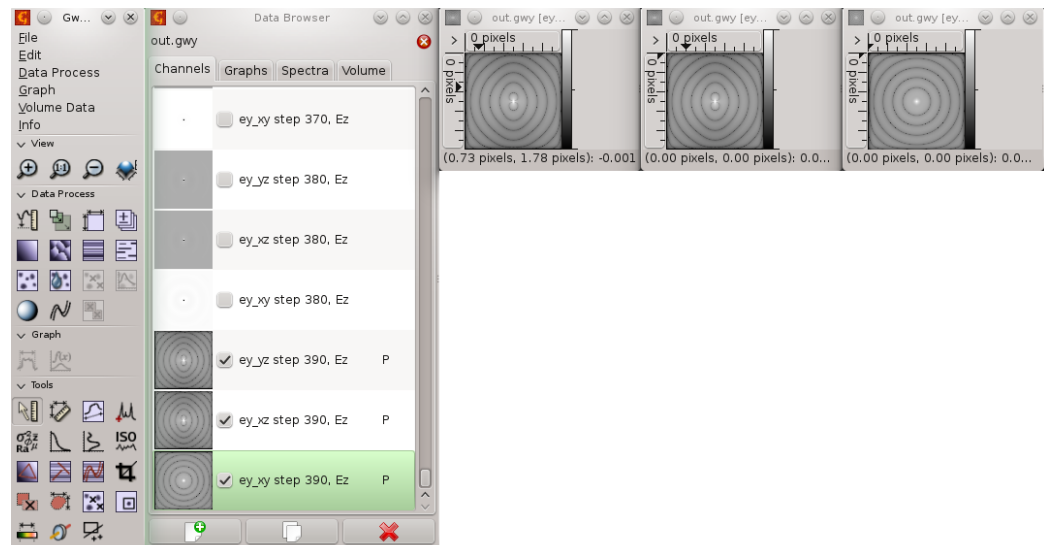
In order to use CPML boundary we click on any of the boundary conditions entries in parameters list. Preset values of CMPL parameters should be fine for this application. Note that setting sigma to -1 leads to automatic estimation of the best value, based discretisation and timestep.

 The image shows a "Boundary settings" dialog box. It has a title bar with standard window controls. The main content is organized into sections. The first section is "Computational volume boundaries:", which lists six boundaries: x0, xn, y0, yn, z0, and zn. Each boundary has a dropdown menu set to "CPML" and a set of four input fields for "CPML parameters (depth, m, σ , a, κ)". The values for these parameters are: depth=10, m=3, σ =-1.000, a=0.030, and κ =4.000. The "a" field for the zn boundary is highlighted with a green border. The second section is "Periodic boundaries inside computational volume:", which contains six checkboxes and input fields: "X0 periodic at" (checked, 1), "XN periodic at" (unchecked, 100), "Y0 periodic at" (checked, 1), "YN periodic at" (unchecked, 100), "Z0 periodic at" (checked, 1), and "ZN periodic at" (unchecked, 100). At the bottom right are "OK" and "Cancel" buttons.

After setting all this we can see the estimated sigma values already in the list of parameter entries.



Then we can **save the file** and **run the computation**. In the output we can see that the wave is damped at the boundaries to zero. Note that CPML is not working only at boundary, but up to some depth (here 10 voxels). This should be considered while adding some objects or sources close to the computational space boundary.

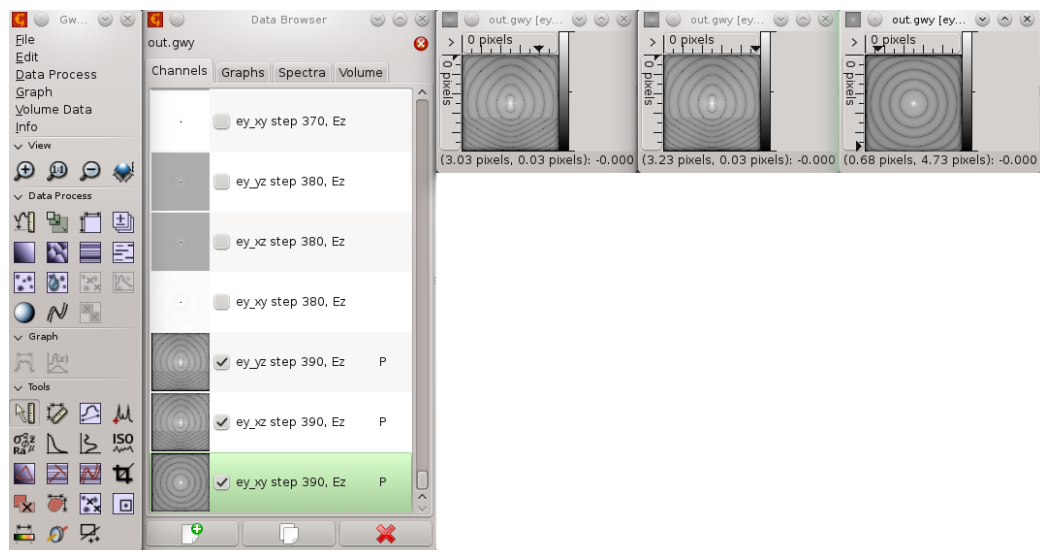


CPML is generally much better absorbing boundary condition than simple second order one (Liao), however it is also much more computationally demanding. Efficiency of its use therefore depends on application.

CPML can work with dielectric media in it, so we can also simulate e.g. halfspace with some permittivity and wave will be damped even there. To test this, click on "vector material file" tab at the bottom, and add this line

```
8 0 0 65 100 100 100 0 3 1 0 0
```

This means adding a rectangular area spanning from 0, 0, 65 to 100, 100, 100, and having relative permittivity of 3, relative permeability 1 and zero losses. Then we save this material file using menu entry *File->Save material file as...* (here we called it halfspace.txt) and we click on some of the media entries in the parameter list to add this file as vector medium. After running the computation we can get this output:



We can see that the wave is damped in both the halfspaces.

Next example

[Back to the documentation.](#)